



National Textile University

Department of Computer Science

Subject:

Operating System

Submitted to:

Sir Nasir Mehmood

Submitted by:

Eman Marium Tariq Rao

Reg number:

23-NTU-CS-1150

Semester:

05

Docker Lab 02

Part 1: Data Persistence with Volumes

Why This Matters:

Before building complex applications, you need to understand how to keep your data safe. Imagine building a blog and losing all posts when you restart!

The Problem: Container Data is Ephemeral

What Happens to Data in Containers?

Let's see what happens to data when a container stops:

```
# Start a container and create some data
docker run -it --name test-container ubuntu bash

# Inside the container, create a file
echo "Important data!" > /data.txt

cat /data.txt # Shows: Important data!

exit

docker rm test-container # Remove the container

# Start a new container
docker run -it --name test-container ubuntu bash

cat /data.txt # Error: No such file!# Data is GONE!
```

```
PS C:\Users\DELL> docker rm test-container
test-container
PS C:\Users\DELL> docker run -it -v my-vol:/home/ubuntu/my-vol --name test-container ubuntu:resolute-20251130 bash
root@a705f40dab18:/#
```

```
PS C:\Users\DELL> docker run -it -v my-vol:/home/ubuntu/my-vol --name test-container ubuntu:resolute-20251130 bash
root@a705f40dab18:/# cd/home/ubuntu/
bash: cd/home/ubuntu/: No such file or directory
root@a705f40dab18:/# cd home/ubuntu/
root@a705f40dab18:/home/ubuntu# ls
my-vol
root@a705f40dab18:/home/ubuntu# cd my-vol/
root@a705f40dab18:/home/ubuntu/my-vol# echo "my-word" >file.txt
root@a705f40dab18:/home/ubuntu/my-vol# cat file.txt
my-word
root@a705f40dab18:/home/ubuntu/my-vol#
```

The problem:

- Container filesystems are temporary
- Data disappears when container is removed
- Can't share data between containers
- Not suitable for databases, uploads, logs, etc.

The Solution: Docker Volumes

Volumes are Docker-managed storage that persists outside containers.

Think of volumes as:

- External USB drives that you can plug into containers
- Persistent storage that survives container deletion
- Shareable between multiple containers

Types of Docker Storage

1. Named Volumes (Recommended for Production)

What: Docker-managed storage in a special location

Where: Docker manages the location (usually `/var/lib/docker/volumes/`) **When:** Databases, persistent app data, production use

```
# Create a named volume
docker volume create my-data

# List volumes
docker volume ls

# Inspect volume details
docker volume inspect my-data

# Use volume in container
docker run -it -v my-data:/home/ubuntu/my-project ubuntu

# Remove volume (only when not in use)
docker volume rm my-data

# Remove all unused volumes
docker volume prune
```

```
PS C:\Users\DELL> docker rm test-container
test-container
PS C:\Users\DELL> docker run -it -v my-vol:/home/ubuntu/my-vol --name test-container ubuntu:resolute-20251130 bash
root@a705f40dab18:/#
```

2. Bind Mounts (For Development)

What: Mount a host directory directly into container

Where: You specify exact host path

When: Development (live code changes), config files

```
# Mount with absolute path
docker run -it -v /home/user/myapp:/home/ubuntu ubuntu
```

```
PS C:\Users\DELL> docker run -it --name my-vol -v C:\Users\DELL\Downloads\eman:/home/ ubuntu:resolute-20251130 bash
root@242ce70af7d4:/# exit
exit
PS C:\Users\DELL> _
```

Part 2: Building Custom Images with Dockerfile

Your First Dockerfile

Let's create a simple custom image step by step!

Step 1: Create a Project Directory

```
cd ~
mkdir my-first-docker-app
cd my-first-docker-app
```

Step 2: Create a Simple Node.js Server

First of all Install `express` in your working directory using `node` .

```
import express from 'express'
const app = express()

app.get('/', (request, response) => {
  response.send('Hello from Node')
})
```

```
app.listen(3000)
```

Code:

```
import express from 'express'
const app = express();

function handler(request, response) {
  response.send('Hello, World!');
}
app.get('/', handler);
app.listen(3000);
```

Terminal:

```
found 0 vulnerabilities
PS D:\semester 5\os\docker> node index.js
PS D:\semester 5\os\docker> |
```

A screenshot of a web browser window. The address bar shows 'localhost:3000'. The page content displays 'Hello, World!' in a simple black font on a white background.

Class Task: Running html file

```
import express from 'express'
const app = express();

// function handler(request, response) {
//   response.send('Hello, World!');
// }
// app.get('/', handler);

app.get('/', (req, res) => {
  res.sendFile(process.cwd() + '/public/index.html');
});

app.use(express.static('public'));
app.listen(4000);
```

```
PS D:\semester 5\os\docker> node index.js
```



Hello world

my name is Eman



Step 3: Create Your First Dockerfile

Create file named `Dockerfile` . Name must be same.

```
# Use Node.js as the base image
FROM node:lts-alpine3.23

# Set working directory inside container
WORKDIR /app

# Copy our application file into the container
COPY index.js .

# Copy package.json
COPY package.json .

# Install dependencies
npm install

# Define the command to run when container starts
CMD ["node", "index.js"]
```

```
from node:lts-alpine3.23

COPY index.js .

COPY package.json .

RUN npm i

CMD ["node", "index.js"]
```

Step 4: Build Your Image

```
docker build -t my-first-app .
```

```
PS D:\semester 5\os\docker> docker build -t my-first-ap .
[+] Building 19.0s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 144B
=> [internal] load metadata for docker.io/library/node:lts-alpine3.23
=> [internal] load .dockerignore
=> [1/4] FROM docker.io/library/node:lts-alpine3.23@sha256:c921b97d4b74f51744057454b306b418cf693865e73b8100559189605
=> => resolve docker.io/library/node:lts-alpine3.23@sha256:c921b97d4b74f51744057454b306b418cf693865e73b8100559189605
=> [internal] load build context
=> => transferring context: 546B
=> [2/4] COPY index.js .
=> [3/4] COPY package.json .
=> [4/4] RUN npm i
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:b2ae1c4cb274d17b8876be9e43e6b4e215ce287c31b00ffa7c999f770fe5d588
=> => exporting config sha256:aabcb0194dd533cfe2499c36d938b3aca4631cc96534f27cb6b50c940cabfa9e
=> => exporting attestation manifest sha256:b94aa9d3d75af4cdb7d911440e60fcf5b0f4f4566fe7e8047860b0e10aaac2c9
=> => exporting manifest list sha256:1a3da22ba6221cf3938ba8498eeedf0c5d7167eff44c2668fe9b628a2569890
=> => naming to docker.io/library/my-first-ap:latest
=> => unpacking to docker.io/library/my-first-ap:latest

1 warning found (use docker --debug to expand):
- ConsistentInstructionCasing: Command 'from' should match the case of the command majority (uppercase) (line 1)
```

Understanding the command:

- `docker build` = build an image
- `-t my-first-app` = tag (name) the image
- `.` = look for Dockerfile in current directory

What you'll see:

Docker processes each instruction and creates layers. Watch the output!

Step 5: Run Your Custom Image

```
docker run my-first-app
```

```
PS D:\semester 5\os\docker> docker run -d -p 3000:3000 my-first-ap
97035694acd84b35c788e648bc0880c6f15a381108cbd84560197be3d34af1dd
PS D:\semester 5\os\docker>
```

Congratulations! You're now a Docker image builder!

Essential Dockerfile Instructions

Let's learn the most important Dockerfile instructions:

1. FROM - Choose Your Base Image

```
# Official Node.js image
FROM node :18
# Node.js with Alpine (smallest)
FROM node :18-alpine
# Official Python image
FROM python :3.9
# Ubuntu
FROM ubuntu :22.04
```

Best Practice: Use official images when possible (verified, maintained)

2. WORKDIR - Set Working Directory

```
FROM node :18-alpine
WORKDIR /app
# Now all commands run from /app
```

3. COPY - Copy Files Into Image

```
# Copy single file
COPY app.js /app/
# Copy entire directory
COPY ./src /app/src
# Copy multiple files
COPY package.json package-lock.json /app/
```

4. RUN - Execute Commands During Build

```
# Install packages
RUN apk add --no-cache curl
# Install Node.js dependencies
RUN npm install
```

5. ENV - Environment Variables

```
# Set environment variables
ENV NODE_ENV=production
ENV PORT=3000
ENV APP_VERSION=1.0.0
# Available at runtime in your application
```

6. EXPOSE - Document Port Usage

```
# Document that container listens on port 80
EXPOSE 80
# Multiple ports
EXPOSE 80 443
```

Important: EXPOSE is documentation only! Still need `-p` flag when running.

7. CMD - Default Command

```
# Exec form (recommended)
CMD ["node", "app.js"]
# Can be overridden when running container
docker run my-app node other_app.js
```

Understanding Layer Caching

This is important for efficient Docker builds!

The Right Way:

```
#  GOOD: Dependencies cached separately
COPY package*.json ./
RUN npm install
COPY app.js .
```

The Wrong Way:

```
#  BAD: Everything rebuilds when code changes
COPY - -
RUN npm install
```

See caching in action:

```
# Change app.js
echo "// Updated" >> app.js
# Rebuild - watch for "CACHED"
docker build -t express-app .
```

Dependencies stay cached! Only `app.js` is recopied. Much faster!

Practice Exercise: Build Your Own Image

Challenge: Serve static HTML files from our previously build node server!

Part 3: Docker Networking

Why It Matters:

After learning how to build images, the next step is making containers talk to each other. Real applications often have multiple services like web servers, databases, and caches that need to communicate.

Why Containers Need Networks

Example scenario:

- Frontend (React) talks to Backend (Express)
- Backend talks to Database (PostgreSQL) and Cache (Redis)

Problem: By default, containers are isolated and cannot see each other.

Solution: Docker Networks!

Containers Are Isolated by Default

```
docker run -d --name app1 alpine sleep 3600
docker run -d --name app2 alpine sleep 3600
docker exec app1 ping app2
# ➤ Fails! Containers cannot communicate by default
```

Custom Networks Solve This

```
# Start two Ubuntu containers on a custom network
docker network create my-network

docker run -d --name app1 --network my-network alpine sleep 3600
docker run -d --name app2 --network my-network alpine sleep 3600

# Ping from one to the other
docker exec app1 ping -c 2 app2
```

Cleanup:

```
docker rm -f app1 app2
docker network rm my-network
```

Types of Docker Networks

1. Bridge Network (Default)

- Private internal network for containers
- Use for most applications

2. Host Network (Advanced)

- Container shares host's network
- Use for special cases needing maximum performance
- Only works on Linux

```
docker run -d --network host nginx
```

Key Takeaways

- Containers on the same network can communicate
 - Container names work as hostnames (DNS)
 - No port mapping is needed for internal communication
 - Networks provide isolation: containers only talk if on the same network
-

Network Management Commands

```
# List networks
docker network ls

# Create network
docker network create my-network

# Inspect network (see connected containers)
docker network inspect my-network

# Connect running container to network
docker network connect my-network my-container

# Disconnect container from network
docker network disconnect my-network my-container

# Remove network
docker network rm my-network

# Remove all unused networks
docker network prune
```

Congratulations!

You've completed Day 2 and learned:

- ✓ **Data Persistence** - Using volumes to keep data safe
- ✓ **Custom Images** - Building your own images with Dockerfile
- ✓ **Docker Networking** - Connecting containers together
- ✓ **Docker Compose** - Orchestrating multi-container applications
- ✓ **Best Practices** - Writing efficient and secure Dockerfiles

Your Docker Journey

What you've accomplished in 2 Labs:

- Lab 1: Running containers, managing images, basic operations
- Lab 2: Building images, connecting services, orchestration

You now have the core skills to:

- Containerize your own applications
 - Set up development environments quickly
 - Share reproducible environments with your team
 - Build multi-container applications
-